

SLIDE 1: TITLE & PROJECT OVERVIEW

DESIGN & IMPLEMENTATION OF A HIGH-FIDELITY PHARMACY INVENTORY & PRESCRIPTION COMPLIANCE DATABASE SYSTEM IN THE KYRGYZ REPUBLIC

COMP2082: Advanced Relational Databases & DBMS Architectures

Presenters: USIT Advanced DBMS Design Lab

Academic Advisor: Prof. Ruslan Aliev, PhD

SLIDE 2: THE PROBLEM STATEMENT

Core Challenges in Community Retail Pharmacy Operations:

- **Inventory Spoilage & Expiry Leakage:** Dispensing expired stock due to lack of batch-level tracking.
- **Prescription Evasion:** Selling regulated drugs (antibiotics/psychotropics) without a valid clinical script.
- **Race Conditions:** Concurrent checkouts double-selling the same physical batch, causing inventory imbalances.
- **Tax Auditing Lag:** Manual calculations of the **standard 12% VAT** in Kyrgyzstan leading to accounting errors.
- **Security Violations:** Standard cashiers accessing staff salary logs or personal identification keys.

SLIDE 3: ENTITY-RELATIONSHIP MODEL (12 NORMALIZED TABLES)

Architectural Layout & Cardinalities:

- **Recursive Hierarchies:** Mapped on `categories(parent_category_id)` to allow multi-level drug classifications.
- **Patients & Employees:** Registered with dates of birth and mandatory **14-digit national PINs** (ПИН/ИИИ).
- **M:N Junction Table Resolution:**
 - `prescription_items` resolves: `prescriptions` $\rightarrow$ `medicines`
 - `sale_items` resolves: `sales` $\rightarrow$ `batches`
- **Audit Logging Ledger:** `inventory_adjustments` logs stock counts, breakages, and supplier returns.

SLIDE 4: PHYSICAL SCHEMA & TYPE SELECTIONS

PostgreSQL 15+ Advanced DDL Implementation:

- **SQL-Standard Identity Columns:** Mapped using `GENERATED ALWAYS AS IDENTITY` to prevent manual sequence bypasses.
- **Native Custom ENUM Domains:** Implements type constraints without small lookup join tables:
 - `biological_gender` ('M', 'F')
 - `employee_role` ('pharmacist', 'manager', 'admin')
 - `payment_method` ('cash', 'card', 'mobile_qr')
 - `prescription_status` ('pending', 'partially_filled', 'filled', 'expired')
 - `adjustment_type` ('breakage', 'spoilage', 'theft', 'reconciliation', 'return_to_supplier')
- **Base Currency:** Formatted using `NUMERIC(12, 2)` in **Kyrgyz Som (KGS)**.

SLIDE 5: RIGOROUS NORMALIZATION (3NF & BCNF)

Eliminating Database Redundancies and Anomaly Vectors:

- **BCNF Proof:** Proven mathematically for core tables (`medicines`, `batches`). Every non-trivial functional dependency $X \twoheadrightarrow Y$ has a determinant X that is a candidate key.
- **Deliberate Denormalization:** `sales` table stores `total_gross`, `tax_amount`, and `total_net`.
 - *Performance Justification:* Prevents expensive aggregate SUM operations during high-frequency reporting.
 - *Auditing Invariance:* Past receipts remain frozen if historical prices change.
 - *Integrity Shield:* Automatically recalculated on line item changes via triggers.

SLIDE 6: PL/PGSQL AUTOMATION ENGINE

Moving Core Business Validation to the Database Layer:

- `tg_deduct_inventory`: Runs on sales checkouts. Locks batch rows via `SELECT FOR UPDATE` to prevent race conditions, validates non-expiration, verifies quantities, and deducts stock.
- `tg_validate_prescription`: Checks prescription-only medicines, validates patient/doctor relationships, verifies remaining limits, and auto-updates prescription status.
- `tg_sale_items_totals`: Automatically extracts the **standard 12% VAT** from gross pricing, updates net amounts, and deducts discounts.
- `tg_sync_inventory_adjustment`: Captures inventory adjustments (wastage, theft) to keep stock levels reconciled.

SLIDE 7: HIGH-PERFORMANCE INDEXING STRATEGY

Tuning Query Latency to Sub-Millisecond Levels:

- **Live Search B-Tree Index:** Composite indexing of trade and generic names to speed up product searches during checkout:

```
CREATE INDEX idx_medicines_search ON medicines (trade_name, generic_name);
```

- **Partial Expiry B-Tree Index:** Optimizes expiry tracking. Excluding depleted batches keeps the index small and lookups fast:

```
CREATE INDEX idx_batches_expiry_tracking ON batches (expiry_date, medicine_id) WHERE (current_quantity > 0);
```

- **Transaction Audits:** Mapped B-Tree indexes on sales timestamps and prescription serial numbers.

SLIDE 8: BI ANALYTICS VIEWS

Real-Time Dashboard Reporting:

- **v_low_stock_medicines**: Tracks stock levels across active batches and calculates recommended reorder quantities.
- **v_expiring_medicines**: Audits batches expiring within 90 days, calculating tied-up capital and potential losses.
- **v_pharmacy_sales_summary**: Provides monthly financial summaries, calculating net revenue, VAT collected, and order values.
- **v_patient_prescription_history**: Traces complete patient histories, linking doctors, prescriptions, and remaining refill limits.
- **v_supplier_medicine_summary**: Analyzes suppliers based on product variety, delivery history, and average acquisition costs.

SLIDE 9: ACID TRANSACTION MODELS & ABORTED ROLLBACKS

Guaranteeing Database State Integrity:

- **Successful Sales Checkout:** Atomic inserts of sales headers and items, with automatic database-level inventory deductions and billing calculations.
- **Inventory Intake:** Reconciles fresh bulk intake with physical stock and logs adjustments in the audit ledger.
- **Trigger-Induced Aborted Rollbacks:** Demonstrates **Atomicity**. If a cashier tries to sell more stock than is available, the triggers raise an exception, aborting the transaction and rolling back all changes (including sales headers).
- **Atomic Clinical Prescribing:** Ensures a prescription and its drug lines are created as a single, indivisible unit to satisfy medical regulations.

SLIDE 10: SECURITY MODEL & PRINCIPLE OF LEAST PRIVILEGE

Group-Level RBAC & Column-Level Protections:

- **Schema Isolation:** Revokes default public access to isolate the schema:

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
```

- **Privilege Assignments:**

- `readonly_auditor`: Group role with read-only (`SELECT`) access across the schema.
- `cashier`: Can check out sales. Restricted from direct stock adjustments or viewing staff security credentials.
- `pharmacist`: Can write prescriptions and check out sales. Has column-level access to the employee directory (excluding passwords and PINs).
- `inventory_manager`: Can manage batches, suppliers, and categories. Restricted from clinical prescriptions and sales checkouts.

SLIDE 11: HIGH-VOLUME SEEDING & QUERIES DEMO

Simulating Real-World Pharmacy Operations:

- **3,000+ logically consistent records** generated via PostgreSQL `DO` blocks.
- **100% active triggers** during seeding, validating the trigger logic and automated calculations.
- **Key BI Queries Supported:**
 - *Dead-Stock Expiration Forecast*: Tracks capital tied up in expiring batches.
 - *Therapeutic Sales Margin Analysis*: Analyzes profitability and COGS by category.
 - *Pharmacist Commission Dashboard*: Calculates average order value (AOV) and cashier sales shares.
 - *Safety Regulatory Auditing*: Audits sales of regulated substances against doctor prescriptions.

SLIDE 12: ARCHITECTURAL REFLECTIONS & PROJECT CONCLUSION

Core Design Lessons & Future Roadmap:

- **Database-Level vs. App-Level Checks:** Implementing validation rules in PL/pgSQL triggers ensures consistent compliance across all client apps (web registers, mobile apps, or APIs).
- **Concurrency Locks:** Using `SELECT FOR UPDATE` is crucial for preventing double-selling race conditions in multi-terminal retail checkouts.
- **Future Improvements:**
 - Integrating HL7/FHIR healthcare data standards.
 - Implementing predictive REST APIs for real-time reporting to the Ministry of Health.
 - Adding machine learning restocking models based on seasonal sales trends.